ADR-18 — Estado e cache de dados no web

Architecture Decision Record · Projeto Steely · Escola de TI · UniCesumar

Status	☒ Proposto ☐ Aceito ☐ Rejeitado ☐ Substituído pelo ADR-___	**Data**	31/10/2026
Responsável	Pietro Pasqual Silva	**Sprint**	8–9 / 15–17
Participantes	Fabrício Notoya Thomas, Augusto Palma Guglielmi, Thiago Lopes Martins, Pietro Pasqual Silva, Rafael Tudela Rizental		
Vínculos	EDTS-46 · EDTS-54 · RNF-04 · RNF-06 · RNF-07 · RNF-09 · ADR-08 · ADR-09 · ADR-13 · ADR-14 · ADR-15 · ADR-17 · ADR-23 · ADR-24		

1. Contexto

obrigatória

Qual é o problema ou a necessidade? Por que decidir agora? Descreva a situação atual e as restrições (prazo, custo, equipe, stack, LGPD).

O front-end web é uma SPA em React + Vite, publicada como site estático na Cloudflare Pages. Todo o tráfego HTTP, inclusive login, passa pela API REST em FastAPI (`/api/v1/...`, OpenAPI 3.1, ADR-13); o cliente não chama Supabase Auth nem PostgREST direto. A conexão direta com o Supabase é só para escutar o Realtime (chat e leilão) com o JWT já emitido (ADR-09). Hoje cada tela trata loading, erro e recarga por conta própria, o que gera requisições duplicadas e estado inconsistente entre telas (ex.: saldo de moedas diferente no header e na carteira).

Restrições: leitura p95 ≤ 200 ms (RNF-04), eventos em tempo real ≤ 100 ms via Supabase Realtime (RNF-06), cache offline de rotas e encontros em IndexedDB definido no ADR-23, guarda de tokens definida no ADR-24, custo zero e entrega até 23/11/2026.

2. Direcionadores da decisão

recomendada

Critérios que pesam na escolha: segurança e privacidade, custo (plano gratuito), esforço até 23/11, facilidade de demonstrar na defesa, manutenção.

- Separar estado do servidor (ofertas, carteira, ESG) de estado de interface (filtros, modais, tema).
- Cache com deduplicação, revalidação e invalidação após mutações.
- Tipos gerados do snapshot do OpenAPI guardado no CI (ADR-13), sem escrever contrato à mão e sem depender da API no ar.
- Integração simples com eventos do Supabase Realtime.
- Compatível com o cache offline do ADR-23 e reaproveitável no app.
- Curva curta para 5 pessoas, pouco boilerplate até 23/11 e biblioteca gratuita.

3. Opções consideradas

obrigatória

Liste no mínimo duas opções, cada uma com descrição, prós e contras.

Opção A — TanStack Query + Zustand

Descrição

TanStack Query cuida do estado do servidor (cache, revalidação, mutações). Zustand guarda o pouco estado global de interface.

Prós

- Pouco código; cache, retry e invalidação prontos
- Persistência opcional via persistQueryClient
- Funciona igual no React Native
- Zustand é minúsculo e sem provider

Contras

- Duas bibliotecas para aprender
- Exige padronizar as chaves de query

Opção B — Redux Toolkit + RTK Query

Descrição

Store única do Redux; RTK Query gera hooks de API e cache a partir de endpoints declarados.

Prós

- Padrão consolidado e previsível
- DevTools completos
- Gerador oficial a partir de OpenAPI

Contras

- Mais boilerplate (slices, store, middlewares)
- Curva de aprendizado maior
- Integração com Realtime exige middleware próprio

Opção C — Context API + SWR

Descrição

Context do React para estado global e SWR (ou fetch manual) para dados remotos.

Prós

- Nenhuma dependência extra (Context)
- SWR é leve e simples

Contras

- Context re-renderiza a árvore inteira
- Menos recursos de mutação e persistência
- Tende a virar código caseiro

ADR-18 — Estado e cache de dados no web

Continuação

4. Decisão

obrigatória

Opção escolhida: ☒ A ☐ B ☐ C. Justifique por que atende melhor aos direcionadores e por que as outras foram descartadas.

TanStack Query v5 para estado do servidor e Zustand para estado de interface, com tipos gerados do OpenAPI do FastAPI.

É a opção com menos código para cumprir cache, revalidação e tempo real, e a mesma camada roda no web e no app. A opção B resolve o problema, mas o boilerplate pesa no prazo de 23/11. A opção C não atende bem à invalidação após mutações nem à persistência.

- Tipos: `openapi-typescript` gera `schema.d.ts` a partir do `openapi.snapshot.json` versionado no CI (ADR-13); o TanStack Query fica sobre o cliente gerado `openapi-fetch`.
- Autenticação: login e renovação do token passam por `/api/v1/auth/` (ADR-13); o cliente HTTP anexa o JWT em `Authorization: Bearer` e, a cada renovação, atualiza o token do Realtime com `realtime.setAuth`, que é o único uso do `supabase-js`.
- Erros: respostas RFC 9457 (Problem Details) viram um tipo único de erro tratado pelo TanStack Query.
- Tempo real: assinaturas do Supabase Realtime (chat e leilão) atualizam o cache com `setQueryData` ou `invalidateQueries`; a escrita continua pelo FastAPI.
- Idempotência (ADR-14): cada ação crítica (lance, gasto e resgate de moedas, confirmação de entrega, Boost, devolução) gera um UUID v4 com `crypto.randomUUID()` no clique e reaproveita a mesma chave em todo retry; `409 IDEMPOTENCY_IN_PROGRESS` espera o `Retry-After`; `Idempotent-Replayed: true` é tratado como sucesso.
- Tela do leilão (ADR-15): `409 BID_OUTDATED` atualiza o leilão no cache e mostra o valor vigente; `409 ALREADY_LEADING` desabilita o botão; a contagem regressiva usa o deslocamento de relógio enviado pelo servidor, que é quem decide o fim.
- Offline: só persistem as queries que o ADR-23 define (rotas e encontro), no IndexedDB escolhido lá; nada de token ou dado pessoal no cache persistido (ADR-24).

5. Consequências

obrigatória

Positivas · Negativas ou riscos aceitos · O que muda no código, no banco, na arquitetura e nos diagramas.

Positivas

- Fim das requisições duplicadas; dados compartilhados entre telas pela mesma chave de cache.
- Mudança de contrato no FastAPI quebra o build do front em vez de quebrar em produção.
- Lances e mensagens aparecem sem refetch manual, via Realtime.

Negativas ou riscos aceitos

- Risco de chaves de query inconsistentes; mitigado com uma fábrica central de chaves.
- Tipos precisam ser regenerados a cada mudança da API; mitigado com script `npm run api:types` no CI.
- Dependência de dois canais (REST e Realtime) para o mesmo dado; o REST é sempre a fonte de verdade.
- Plano gratuito do Realtime limita a 200 conexões e 100 msg/s; as 1.000 conexões do RNF-06 exigem o plano Pro sem teto de gastos (o Pro padrão dá 500), sem mudança de código (ADR-15).

O que muda

- Build: Vite gera site estático publicado na Cloudflare Pages (deploy automático pelo GitHub e preview por pull request), sem servidor para hibernar. Código: `src/api` com `schema.d.ts` gerado, `client.ts` (openapi-fetch + JWT), `queryKeys.ts` e hooks por domínio (`useOffers`, `useWallet`, `useEsg`); `src/realtime` com as assinaturas; `src/stores` com stores Zustand de UI.
- Arquitetura: `QueryClientProvider` na raiz; `staleTime` padrão de 30 s e 0 s para carteira e leilões.
- Diagramas: o diagrama de componentes do front mostra FastAPI (REST) e Supabase Realtime como as duas fontes do cache.

6. Validação

recomendada

Como saber se a decisão deu certo? Teste, critério de aceite, métrica ou demonstração que comprova.

- Navegar entre listagem e detalhe de oferta não dispara nova requisição dentro do `staleTime` (DevTools e aba Network).
- Após um lance, o valor atualiza em outra aba aberta em ≤ 1 s, sem recarregar (Realtime → cache).
- Alterar um schema Pydantic, atualizar o snapshot e regenerar os tipos faz o `tsc` apontar os usos quebrados no mesmo pull request.
- Reenviar um lance com a mesma `Idempotency-Key` após queda de rede não cria segundo lance.
- Testes de hooks com Vitest + Testing Library, dentro da pirâmide do ADR-17.

7. Plano de implementação

recomendada

Tarefas necessárias, responsáveis e sprint prevista.

Tarefa	Responsável	Sprint
Configurar o projeto Vite e o deploy na Cloudflare Pages	A definir	A definir
Instalar TanStack Query, Zustand, openapi-typescript e openapi-fetch	A definir	A definir
Gerar tipos de openapi.snapshot.json e criar client.ts com JWT (EDTS-46)	A definir	A definir
Criar queryKeys.ts e hooks de ofertas e carteira; migrar telas (EDTS-54)	A definir	A definir
Assinar canais do Supabase Realtime (chat e leilão) e ligar ao cache	A definir	A definir
Ativar persistência só das queries definidas no ADR-23	A definir	A definir
Geração e reuso da Idempotency-Key no web (ADR-14)	Pietro	A definir
Tela do leilão: tratamento de 409 e contagem regressiva (ADR-15)	Pietro	A definir

8. Referências

recomendada

Documentação, artigos, diagramas e outros ADRs.

- Vite — vite.dev
- Cloudflare Pages — developers.cloudflare.com/pages
- TanStack Query — tanstack.com/query/latest
- Zustand — zustand.docs.pmnd.rs
- openapi-typescript / openapi-fetch — openapi-ts.dev
- Supabase Realtime — supabase.com/docs/guides/realtime
- ADR-08 (banco), ADR-09 (auth e Realtime), ADR-13 (contrato), ADR-14 (idempotência), ADR-15 (leilão), ADR-17 (testes), ADR-23 (offline), ADR-24 (tokens)

Histórico de revisões

Versão	Data	Autor	Alteração
1.0	31/10/2026	Pietro Pasqual Silva	Criação do ADR
1.1	31/10/2026	Pietro Pasqual Silva	Alinhamento com as decisões do grupo (FastAPI, Supabase, pg_cron)
1.2	31/10/2026	Pietro Pasqual Silva	Fechamento dos conflitos do alinhamento (Vite, snapshot OpenAPI, RFC 9457, RLS no FastAPI)
1.3	31/10/2026	Pietro Pasqual Silva	Segundo alinhamento: limites do Realtime, slowapi, SECURITY DEFINER e sessão RLS
1.4	31/10/2026	Pietro Pasqual Silva	Hospedagem do front definida: Cloudflare Pages
1.5	31/10/2026	Pietro Pasqual Silva	Alinhamento com os ADR-13 a 17 (codes, camelCase, schema private, Idempotency-Key, tela do leilão)
1.6	01/11/2026	Revisão cruzada do grupo	Padronização do nome 'Fabrício' nos participantes, conforme a revisão cruzada.

ADENDO — ADR-18 · Estado e cache de dados no web

Projeto Steely · Alinhamento final (propostas do Augusto aceitas). Itens do grupo aplicados a este documento.

Cronograma (item 18)

- Janela de entrega (item 18): Sprints 8–9 / 15–17, até 23/11/2026.
- O campo Sprint do cabeçalho passa a registrar essa janela.

Escopo de telas do web (item 12)

- Aviso de "saldo simulado" (ADR-03).
- Fila de dupla aprovação do moderador (ADR-06).
- Fluxo de exclusão de conta com carência de 30 dias e comprovante (ADR-07).
- Aviso de "volume insuficiente para o recorte", com sugestão de nível superior no DaaS (ADR-04).
- Painel de consulta de logs (STY056, ADR-05).
- Cada tela consome dados por hooks com chaves próprias em queryKeys.ts, seguindo a seção 4 deste ADR (TanStack Query).

Desligamento após a banca (item 19)

- Após a banca, o front deixa de chamar o endpoint fast-forward-timers e deixa de exibir dados rotulados "DEMONSTRAÇÃO". O registro desse desligamento fica no ADR-16.

Histórico: versão 1.7 — aplicação dos itens 12, 13, 18 e 19 do alinhamento final do grupo. Este adendo registra as decisões de negócio do grupo que afetam este ADR; o texto das seções 1 a 8 permanece como na versão anterior.